

Name:

Hemos ID:

Student ID (학번):

CSED-101 PROGRAMMING & PROBLEM SOLVING

Spring 2025 (Mock Test)

Finalterm

	Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8	Q9	Q10	Q11	Q12	Total
Your Score													
Max Score	3	7	9	10	8	11	10	8	9	12	7	6	100

- Write down your name, hemos ID, and student ID.
- There are 13 pages in this final.
- Your answers must run correctly in **Python programming language** without error or warning. Otherwise, your answers will be considered incorrect. For example, it is ok to put more parentheses than needed in your answer, but it will be incorrect if you put fewer parentheses than needed.
- You must write your answer on the underline => _____. Scratches outside the underline will be ignored.
- The total score is 100
- This is a 2-hour exam.

명예서약서

나는 명예로운 포스테키안으로서 다음의 Honor Code를 지킬 것을 맹세합니다.

정직과 타인에 대한 존중이 함께하는 포스테키안의 미래는 명예롭다.

2025년 월 일
이름 (인)

1. (3점) 아래 문항에 올바르게 답하시오. 다만, 코드에서 오류가 날 경우, ERROR라고 작성할 것.

1.1. (1점) 다음 코드의 결과를 작성하시오.

<pre>class Animal: def speak(self): print("동물이 소리를 낸다.") class Dog(Animal): def speak(self): print("멍멍!") class Cat(Animal): def speak(self): print("야옹~") Dog().speak() Cat().speak()</pre>	답안 입력
---	-------

1.2. (2점) 다음 코드의 결과를 작성하시오.

<pre>class Animal: types = "Dog" def __init__(self, name): self.name = name print(Animal.types) dog1 = Animal("Buddy") print(dog1.types) dog1.types = "Cat" print(dog1.types)</pre>	답안 입력
---	-------

2. (7점) **균형 잡힌 괄호**란 괄호가 열린 만큼 닫히고, 올바른 순서로 닫혀 있는 경우를 말한다. 즉, 아래의 예시와 같이 왼쪽과 오른쪽의 짝이 맞는 괄호를 균형 잡힌 괄호라고 부른다.

균형 잡힌 괄호의 예시: (), (()), ()(), ((()())), [()]

다음은 스택을 이용하여 입력받은 값이 **균형 잡힌 괄호**인지 여부를 확인하는 코드의 일부이다. 프로그램이 올바르게 작동하도록 빈칸을 채우시오.

```
opening = "{["
closing = "}]]"
matching = {'(': ')', '[': ']'

def is_matched_parentheses(text):
    stack = []

    for char in text:
        if char in opening:
            -----

        elif char in closing:
            if -----:
                -----

    return -----
```

3. (9점) 다음은 클래스를 활용하여 **학생**을 추상화한 것이다. 아래 조건을 참고하여, 코드가 올바르게 되도록 빈칸을 채우시오.

- **Student**: 이름(name)과 성적(grade)을 인스턴스 변수로 가지며, 인스턴스의 값을 출력하는 display 메서드를 갖는 클래스이다.
- **Undergraduate**: Student를 상속받아 생성되며, 전공(major)이 추가된 클래스이다.

```
class Student:
    def __init__(self, -----):
        -----
        -----

    def display(self):
```

```

        print(f"이름: {_____}, 성적: {_____}")

class Undergraduate(Student):
    def __init__(self, _____):
        _____
        _____

    def display(self):
        _____ # print를 쓰지 않고 작성할 것.
        print(f"전공: {_____}")

student = Student("홍길동", 90)
student.display()

under = Undergraduate("박태준", 85, "컴퓨터공학")
under.display()

```

4. (10점) 다음은 파이썬 클래스를 사용하여 **계좌**를 추상화한 것이다. 아래의 조건을 참고하여 빈칸을 채우시오.

- Account: 계좌를 추상화한 클래스로 소유자 이름과 초기 잔액을 인자로 받아 초기화한다.
- SavingAccount: 출금 기능이 비활성화된 계좌 클래스이다.
- CheckingAccount: 출금 기능이 가능하되, 잔액보다 큰 금액은 출금할 수 없는 계좌 클래스이다.
- 모든 통장의 잔액은 직접 확인하거나 수정할 수 없어야 한다.

```

class Account:
    def __init__(self, owner, initial_balance):
        _____
        _____

    def withdraw(self, amount): # 잔액을 감소시키는 메서드.
        pass

    def deposit(self, amount): # 잔액을 증가시키는 메서드. 증가액은 양수만 허용되어야 한다.
        if _____:
            _____
        else:

```

```

        print("잘못된 입금 금액입니다.")

def _get_balance(self):
    return _____

def _update_balance(self, new_balance):
    _____

def get_owner(self):
    return _____

class SavingAccount(Account):
    _____
    print("적금 통장에서는 출금이 불가능합니다.")

class CheckingAccount(Account):
    def withdraw(self, amount):
        if _____:
            _____
        else:
            print("잔액이 부족합니다.")

```

5. (8점) 다음은 파이썬 클래스를 활용하여 복소수를 구현하는 예제이다. 조건에 만족하게 코드를 완성하시오.

- 생성자는 복소수의 실수부(real)와 허수부(imag)를 받는다. 생성자가 숫자를 하나만 받는다면 허수부가 0이 되도록 초기화한다.
- 복소수끼리의 덧셈, 복소수와 상수의 덧셈 연산이 모두 가능하다. 이때, 덧셈 연산은 양방향을 지원해야 한다. 즉, $a+b=b+a$ 가 되어야 한다.
- 클래스의 인스턴스에 `len()` 함수를 사용하면, 해당 복소수의 절댓값 (크기, $\sqrt{a^2 + b^2}$)을 반환한다.
- 클래스의 인스턴스를 `print()`로 출력할 경우, "**a + bi**" 혹은 "**a - bi**"의 형식으로 출력되어야 한다.
- 현 예제에서 복소수와 상수, 복소수와 복소수의 덧셈 이외의 덧셈은 수행하지 않는다.

힌트: `math` module의 `sqrt` 함수는 실수 1개를 입력받아 제곱근을 출력한다. 또한, 같은 모듈의 `fabs` 함수는 실수 1개를 입력받아 절댓값을 출력한다.

```
import math
```

```

class ComplexNumber:
    def __init__(self, _____):
        self.real = _____
        self.imag = _____

    def _____(self, other): # 덧셈
        if isinstance(other, ComplexNumber):
            return ComplexNumber(_____, _____)
        else: return ComplexNumber(_____, _____)

    def _____(self, other): # 실수 반대쪽 덧셈
        return ComplexNumber(_____, _____)

    def __len__(self): # 절댓값 반환
        return _____

    def __str__(self): # 값 print 함수에서의 출력
        sign = '+' if self.imag >= 0 else '-'
        return f"_____ "

```

6. (11점) 다음 질문은 정렬과 관련한 내용을 담고 있다. 물음에 답하시오.

6.1. (3점) 다음과 같은 학생 점수 리스트가 주어졌다고 하자.

```
students = [("Kim", 90), ("Lee", 75), ("Park", 95), ("Choi", 80)]
```

해당 리스트에서 학생들의 점수를 기준으로 오름차순, 내림차순으로 각각 정렬하시오.

```

# 점수를 기준으로 오름차순 정렬
sorted_students = _____(students, key=_____)

# 점수를 기준으로 내림차순 정렬 (원본 리스트 변경)
students._____ (key=_____, reverse=True)

```

6.2. (5점) 아래 코드는 **버블 정렬**을 구현한 것이다. 빈칸을 채워 정렬 알고리즘을 완성하시오.

```

def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(_____):

```

```
if _____:
    _____
```

6.3. (3점) 아래 리스트를 선택 정렬, 삽입 정렬을 통해 각각 **오름차순 정렬**(오른쪽 원소가 더 크게 되도록 정렬)하고 그 결과를 작성하시오. 단, 각 정렬 단계별 정렬 결과를 작성해야 한다.

- (선택 정렬) 입력 리스트 1: [5, 4, 2, 3, 1]
- (삽입 정렬) 입력 리스트 2: [4, 3, 5, 1, 2]

선택 정렬 과정	삽입 정렬 과정

7. (10점) 다음 질문들은 머지 정렬과 관련한 질문을 담고 있다. 물음에 답하시오.

7.1. (5점) 다음은 머지 정렬(merge sort)을 구현하는 코드이다. 빈칸을 채워 해당 정렬 코드를 완성하여라.

```
def merge_sort(arr):
    if _____:
        _____

    mid = _____
    left = _____
    right = _____

    return _____

def merge(left, right):
    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i])
```

```

        i += 1
    else:
        result.append(right[j])
        j += 1

    result += left[_____]
    result += right[_____]
    return result

```

- 7.2. (5점) 다음 리스트가 위에서 작성된 머지 정렬 알고리즘에 의해 정렬된다고 하자. 이때, 그 정렬 과정과 정렬 결과를 작성하시오.

```

data = [5, 4, 6, 1, 2, 7, 3]
sorted_data = merge_sort(data)

```

머지 정렬 과정

8. (8점) 다음은 정렬된 배열에서 특정 값을 탐색하는 이진 검색 코드의 일부이다. 빈칸을 채워 알맞게 코드를 완성하여라.

```

def binary_search(arr, target):
    left = _____
    right = _____

    while _____:
        mid = _____

        if arr[mid] == target:
            return True
        elif arr[mid] < target:
            left = _____
        else:

```

```
right = _____
```

```
_____
```

9. (9점) 수열 $A(n)$ 이 다음과 같이 정의되어 있다고 하자.

$$A(n) = A(n-1) + 2 \cdot A(n-2), \quad A(0) = 1, \quad A(1) = 2$$

9.1. (4점) 아래의 코드는 이 수열을 재귀적으로 계산하는 코드의 일부이다. 주어진 코드를 **재귀**(recursion)를 이용하여 완성하여라.

```
def sequence(n):
    if n == 0:
        return _____
    elif n == 1:
        return _____
    else:
        return _____
```

9.2. (5점) 수열을 재귀적으로 계산할 때, 같은 수열 값이 여러 번 계산되는 것을 확인할 수 있다. 이때, 메모이제이션이라는 기술을 도입하면 이를 방지할 수 있다. **메모이제이션**(memoization)이란, 이전 수열의 값을 배열에 따로 작성하여 새로운 값을 구할 때 불필요한 재귀를 줄이는 않는 기술이다. 다음은 같은 수열을 메모이제이션을 사용하여 효율을 높인 것이다. 코드를 완성하여라.

```
memo = {}

def sequence_memo(n):
    if n in _____:
        return _____
    if n == 0:
        memo[n] = 1
    elif n == 1:
        memo[n] = 2
    else:
        memo[n] = _____

    return memo[n]
```

10. (12점) 하노이 탑 문제는 세 개의 기둥과 여러 개의 크기 다른 원판이 있을 때, 모든 원판을 다음 조건을 만족하면서 한 기둥에서 다른 기둥으로 옮기는 최소 횟수를 구하는 문제이다.

조건

- (a) 한 번에 하나의 원판만 옮길 수 있다.
- (b) 큰 원판이 작은 원판 위에 쌓일 수 없다.
- (c) 세 개의 기둥(A, B, C)을 사용하여 모든 원판을 A에서 C로 옮긴다.

- 10.1. (4점) 아래는 하노이 탑 문제를 재귀적으로 해결하는 함수를 구현하는 예제이다. 빈칸을 채워 코드를 완성하여라.

```
def hanoi(n, _____):  
    if n == 1:  
        print(f"Disk 1: {_____} -> {_____}")  
        return  
    hanoi(_____, _____)  
    print(f"Disk ____: {_____} -> {_____}")  
    hanoi(_____, _____)
```

- 10.2. (3점) 위와 같이 재귀적으로 정의된 하노이 함수에 대해, `hanoi(n, 'A', 'C', 'B')` 를 호출한다. $n = 3$ 인 경우의 출력 결과를 작성하시오.

출력 결과

- 10.3. (5점) 모든 재귀 함수는 함수 호출을 기반으로 작성된다. 함수의 호출은 그 함수가 호출된 위치가 우선 스택에 저장된 뒤, 나중에 함수가 완료되면 스택에서 다시 원소를 하나씩 꺼내어 기존 위치로 되돌아가는 방식으로 동작한다. 즉, 재귀 함수로 구현할 수 있는 모든 문제는 스택으로도 동일하게 구현할 수 있다. 아래 코드는 하노이 탑 문제를 스택을 이용하여 **재귀 없이 반복문**으로 구현한 것이다. 이를 위해 `collections` 모듈을 이용하였다. 빈칸을 채워 코드를 완성하여라.

```
from collections import deque  
  
def hanoi_iterative(n, _____):  
    stack = deque()  
    stack.append((_____, _____))
```

```

while stack:
    _____ = stack.pop()

    if n == 1:
        print(f"Move disk 1 from {_____} -> {_____}")
    else:
        stack.append((_____, _____))
        stack.append((_____, _____))
        stack.append((_____, _____))

```

11. (7점) 다음 질문들은 numpy, pandas와 관련한 질문을 담고 있다. 물음에 답하시오.

11.1. (4점) 다음은 numpy 배열을 활용한 기초적인 통계 및 배열 조작 예제이다. 빈칸을 채워 코드를 완성하여라.

```

import numpy as np

data = np.array([1, 2, 3, 4, 5, 6])

mean = np._____(data)
std = np._____(data)
print(data._____) # 2x3 모양의 numpy 배열을 출력한다.

```

11.2. (3점) 아래는 pandas를 이용해 학생 점수 정보를 데이터프레임으로 처리하는 예시이다. 이때, '총점'은 '국어' 점수와 '수학' 점수의 합으로 계산된다. 빈칸을 채워 코드를 완성하여라.

```

import pandas as pd

data = {
    '이름': ['철수', '영희', '민수'],
    '국어': [90, 80, 70],
    '수학': [85, 95, 60]
}

df = _____(data)

average_kor = _____ # 국어 점수 평균을 계산한다.
df['총점'] = _____

```

12. (6점) 다음은 숫자 두 개를 클릭하면 그 합을 계산하고, 막대기 텍스트로 이를 보여주는 **어린이 교육용 GUI 프로그램**이다. 합이 5 이상이면, 5개 막대마다 ||||/ 형태로 묶어 텍스트를 표현하여야 한다. 아래 코드의 빈칸을 채워 프로그램을 완성하여라.

```
import tkinter as tk

class StickAdder:
    def __init__(self, root):
        self.root = root
        self.root.title("막대기 덧셈 놀이")
        self.selected = []

        self.label = tk.Label(root, text="숫자를 두 개 선택하세요")
        self.label.pack()

        self.buttons_frame = tk.Frame(root)
        self.buttons_frame.pack()

        for i in range(1, 10):
            btn = tk.Button(self.buttons_frame, text=str(i), width=4,
                            command=_____ )
            btn.grid(_____) # 버튼 배열은 3x3 되어야 한다.

        self.result_label = tk.Label(root, text="")
        self.result_label.pack()

    def select_number(self, number):
        if len(self.selected) < 2:
            self.selected.append(number)
            self.label.config(text=f"선택된 숫자: {self.selected}")
        elif len(self.selected) == 2:
            total = _____(self.selected)
            sticks = self.to_sticks(total)
            self.result_label.config(text=f"_____")
            self.selected.clear()
            self.label.config(text="숫자를 두 개 선택하세요")
```

```

def to_sticks(self, number):
    return _____

if __name__ == "__main__":
    root = _____ # 메인 화면을 생성한다.
    app = StickAdder(root)
    _____ # root의 loop를 시작한다.

```